# Triggers from past exams

# TrickTruck (September 12th, 2023)

A logistics company manages shipments via trucks. Each driver owns one or more trucks and has a number of colleagues who can drive his/her trucks. A truck makes zero or more trips. A trip specifies that a given truck must travel from an origin to a destination on a certain date. When the driver completes a trip, a report is created with the number of hours travelled. Given the following logical schema (keys in uppercase letters):

TRUCK(PLATE, status, ownerId) – status: available (default), maintenance

DRIVER(ID, hoursTravelled, status) – status: authorized (default), non-authorized

– hoursTravelled defaults to 0

TRIP(TRIPID, date, origin, destination, truckPlate, driverId)

TRIPREPORT(TRIPID, hoursTravelled)

ALTERNATE(DRIVERID, ALTERNATEDRIVERID) – DRIVERID is the owner

Suppose that when a new trip is created, the initial values for the truckPlate and driverId are set to NULL.

# TrickTruck (September 12th, 2023)

Design a set of triggers that implement the following behaviors:

1) When a truck is assigned to a trip, if the status of the truck is maintenance the operation is prevented from proceeding and the trigger raises an exception with text Truck under maintenance. After the assignment of an available truck to a trip, a trigger tries to automatically assign a suitable driver, giving priority to the truck's owner. He/she can be assigned only if his/her status is authorized and he/she is available for that date. Otherwise, an alternate driver is searched: the one with the least cumulative travel hours among those with authorized status available for that date is chosen. If a suitable driver is not found, the trigger raises an exception with text No driver available.

2) When a trip report is created, the total number of hours travelled by the driver must be updated. If the hours s/he has travelled exceeds 1000, his/her status is set to non-authorized and his/her assignments to the trips posterior to the reported one are set to NULL.

When a truck is assigned to a trip, if the status of the truck is maintenance the operation is prevented from proceeding and the trigger raises an exception with text Truck under maintenance.

TRUCK(PLATE, status, ownerId)
DRIVER(ID, hoursTravelled, status)
TRIP(TRIPID, date, origin, destination, truckPlate, driverId)

```
CREATE TRIGGER CheckMaintenance
BEFORE UPDATE OF truckPlate ON TRIP
FOR EACH ROW
BEGIN
        DECLARE truck_status VARCHAR(20);

        SELECT status INTO truck_status
        FROM TRUCK
        WHERE PLATE = NEW.truckPlate;
        IF truck_status = 'maintenance' THEN
                SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Truck under maintenance';
        END;
END;
```

After the assignment of an available truck to a trip, a trigger tries to automatically assign a suitable driver, giving priority to the truck's owner. He/she can be assigned only if his/her status is authorized and he/she is available for that date. Otherwise, an alternate driver is searched: the one with the least cumulative travel hours among those with authorized status available for that date is chosen. If a suitable driver is not found, the trigger raises an exception with text No driver available.

TRUCK(PLATE, status, ownerId)

DRIVER(ID, hoursTravelled, status)

TRIP(TRIPID, date, origin, destination, truckPlate, driverId)

After the assignment of an available truck to a trip, a trigger tries to automatically assign a suitable driver, giving priority to the truck's owner.

TRUCK(PLATE, status, ownerId)
DRIVER(ID, hoursTravelled, status)
TRIP(TRIPID, date, origin, destination, truckPlate, driverId)

```
CREATE TRIGGER AssignDriver
AFTER UPDATE OF truckPlate ON TRIP
FOR EACH ROW
BEGIN
        DECLARE owner_id INT;
        DECLARE suitable_driver_id INT;

        SELECT ownerId INTO owner_id
        FROM TRUCK
        WHERE PLATE = NEW.truckPlate;
        ...
```

*Get the owner*

*Continues…*

After the assignment of an available truck to a trip, a trigger tries to automatically assign a suitable driver, giving priority to the truck's owner.

TRUCK(PLATE, status, ownerId)
DRIVER(ID, hoursTravelled, status)
TRIP(TRIPID, date, origin, destination, truckPlate, driverId)

```
...
SELECT ID INTO suitable_driver_id
FROM DRIVER
WHERE ID = owner_id AND status = 'authorized'
AND NOT EXISTS (
        SELECT *
        FROM TRIP T
WHERE T.driverId = owner_id
AND T.date = NEW.date
)
...
```

*Try to assign the owner as driver if authorized and available*

*Check if the driver is available for that date*

*Continues…*

Otherwise, an alternate driver is searched: the one with the least cumulative travel hours among those with authorized status available for that date is chosen

```
        ...
IF suitable_driver_id IS NULL THEN
        SELECT A.ALTERNATEDRIVERID INTO suitable_driver_id
        FROM ALTERNATE A
        JOIN DRIVER D ON A.ALTERNATEDRIVERID = D.ID
        WHERE A.DRIVERID = owner_id
        AND D.status = 'authorized'
        AND NOT EXISTS (
                SELECT *
                FROM TRIP T
                WHERE T.driverId = A.ALTERNATEDRIVERID
                AND T.date = NEW.date
                )
        ORDER BY D.hoursTravelled ASC
        LIMIT 1;
END IF;
        ...
```

*If the owner is not available or authorized*

*Find an alternate driver*

*Check if the driver is available for that date*

TRUCK(PLATE, status, ownerId)
DRIVER(ID, hoursTravelled, status)
TRIP(TRIPID, date, origin, destination, truckPlate, driverId)
ALTERNATE(DRIVERID, ALTERNATEDRIVERID)

*Continues…*

If a suitable driver is not found, the trigger raises an exception with text No driver available.

```
        ...
        IF suitable_driver_id IS NOT NULL THEN
                UPDATE TRIP
                SET driverId = suitable_driver_id
                WHERE NEW.TRIPID = TRIPID;
        ELSE
                SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'No driver available';
        END IF;
END IF;
END;
```

*If found, set the driver for the trip*

*Check if the driver is available for that date*

TRUCK(PLATE, status, ownerId)
DRIVER(ID, hoursTravelled, status)
TRIP(TRIPID, date, origin, destination, truckPlate, driverId)
ALTERNATE(DRIVERID, ALTERNATEDRIVERID)

When a trip report is created, the total number of hours travelled by the driver must be updated. If the hours s/he has travelled exceeds 1000, his/her status is set to non-authorized and his/her assignments to the trips posterior to the reported one are set to NULL.

TRUCK(PLATE, status, ownerId)
DRIVER(ID, hoursTravelled, status)
TRIP(TRIPID, date, origin, destination, truckPlate, driverId)
TRIPREPORT(TRIPID, hoursTravelled)
ALTERNATE(DRIVERID, ALTERNATEDRIVERID)

```
CREATE TRIGGER UpdateDriverHours
AFTER INSERT ON TRIPREPORT
FOR EACH ROW
BEGIN
        UPDATE DRIVER
        SET hoursTravelled += NEW.hoursTravelled
        WHERE ID = (SELECT driverId FROM TRIP WHERE TRIPID =
NEW.TRIPID);
END;
```

When a trip report is created, the total number of hours travelled by the driver must be updated. If the hours s/he has travelled exceeds 1000, his/her status is set to non-authorized and his/her assignments to the trips posterior to the reported one are set to NULL.

TRUCK(PLATE, status, ownerId)
DRIVER(ID, hoursTravelled, status)
TRIP(TRIPID, date, origin, destination, truckPlate, driverId)
TRIPREPORT(TRIPID, hoursTravelled)
ALTERNATE(DRIVERID, ALTERNATEDRIVERID)

```
CREATE TRIGGER UpdateDriverStatus
AFTER UPDATE OF hoursTravelled ON DRIVER
FOR EACH ROW
WHEN NEW.hoursTravelled > 1000
BEGIN
        UPDATE DRIVER
        SET status = 'non-authorized'
        WHERE ID = NEW.ID;

        UPDATE TRIP
        SET driverId = NULL
        WHERE driverId = (SELECT driverId FROM TRIP WHERE TRIPID = NEW.TRIPID
        AND date > NOW();
END;
```

When a trip report is created, the total number of hours travelled by the driver must be updated. If the hours s/he has travelled exceeds 1000, his/her status is set to non-authorized and his/her assignments to the trips ***posterior to the reported one*** are set to NULL.

- *Alternative solution (single trigger)*

TRUCK(PLATE, status, ownerId)
DRIVER(ID, hoursTravelled, status)
TRIP(TRIPID, date, origin, destination, truckPlate, driverId)
TRIPREPORT(TRIPID, hoursTravelled)
ALTERNATE(DRIVERID, ALTERNATEDRIVERID)

```
CREATE TRIGGER UpdateDriverStatus
AFTER INSERT ON TRIPREPORT
FOR EACH ROW
BEGIN
    DECLARE oldHours INT;

    SELECT hoursTravelled INTO oldHours
    FROM DRIVER
    WHERE ID = (SELECT driverId FROM TRIP WHERE TRIPID = NEW.TRIPID);

     UPDATE DRIVER
    SET hoursTravelled += NEW.hoursTravelled
    WHERE ID = (SELECT driverId FROM TRIP WHERE TRIPID = NEW.TRIPID);

    IF oldHours + NEW.hoursTravelled > 1000 THEN
        UPDATE DRIVER
        SET status = 'non-authorized'
        WHERE ID = (SELECT driverId FROM TRIP WHERE TRIPID = NEW.TRIPID);

        UPDATE TRIP
        SET driverId = NULL
        WHERE driverId = (SELECT driverId FROM TRIP WHERE TRIPID = NEW.TRIPID)
            AND date > (SELECT date FROM TRIP WHERE TRIPID = NEW.TRIPID);
    END IF;
END;
```

*Driver.status updated ONLY after an insert on TripReport, not for every update of its hours*

# DB2 Exam (January 14th, 2022)

The **interval(<u>LeftEnd,RightEnd</u>)** table contains two (non-negative) integer attributes representing the ends of closed time intervals.

The **covered(<u>upTo</u>)** table contains a single row with a single integer (initially set to 0) that keeps track of the longest time span from 0 covered by one or more intervals (i.e., all instants from 0 to upTo are in some interval in interval).

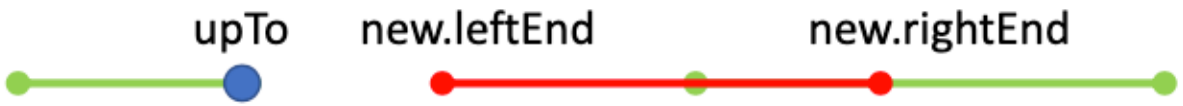
For instance, after inserting [0,10], [10,15], and [14,21] into interval, the value of upTo is updated to 21; if the intervals [25,29], [28,31], and [20,26] are also added (in any order), then upTo should be updated to 31; if [35,38] is then added, upTo remains unchanged. Users can only insert new time intervals.

1) Design a trigger system that updates upTo;

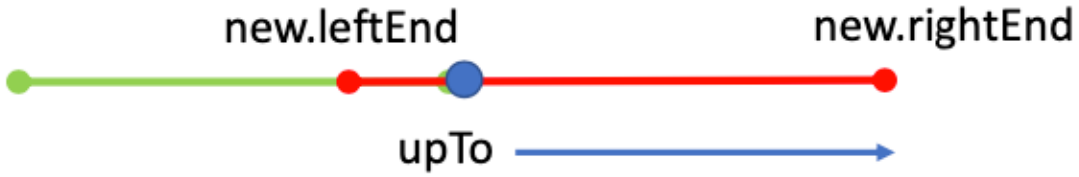
2) Discuss termination.

New interval in red,
Already existing intervals in green

## 1) No update is needed:




## 2) Need to update:



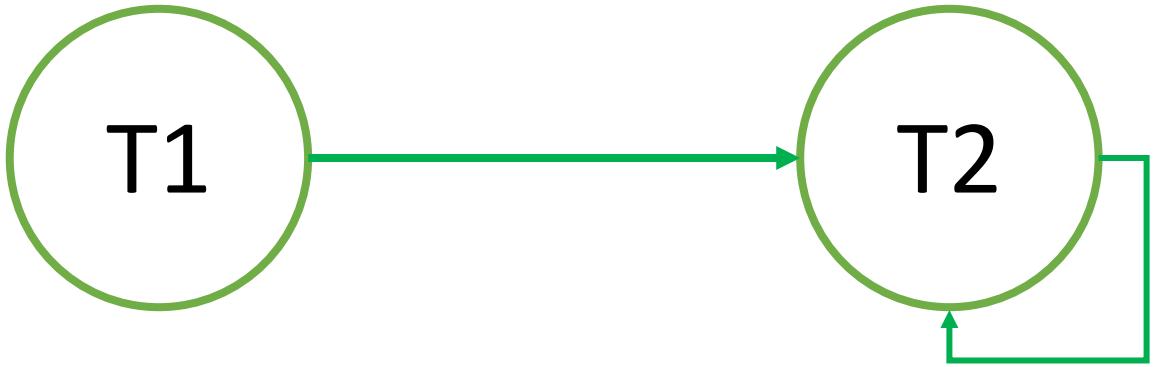
**interval(<u>LeftEnd,RightEnd</u>)**
**covered(<u>upTo</u>)**

```
CREATE TRIGGER ProcessNewInterval
AFTER INSERT ON interval
FOR EACH ROW
BEGIN
    IF (new.LeftEnd <= (SELECT upTo FROM covered)
        AND new.RightEnd > (SELECT upTo FROM covered)) THEN
            UPDATE covered SET upTo = new.RightEnd;
    END IF;
END
```

**interval(LeftEnd,RightEnd)**
**covered(upTo)**

```
CREATE TRIGGER UpdateCovered
AFTER UPDATE ON covered
FOR EACH ROW
        DECLARE possibleRightEnd INTEGER;
        SELECT coalesce(max(RightEnd),new.upTo) INTO possibleRightEnd
        FROM interval
        WHERE LeftEnd <= new.upTo AND RightEnd > new.upTo;
        IF possibleRightEnd > new.upTo THEN
                UPDATE covered SET upTo = possibleRightEnd;
        END IF;
```

*max(RightEnd) can be NULL for this WHERE clause*




Although the triggering graph is cyclic, since UpdateCovered (T2) can recursively activate itself, the conditions *RightEnd* > *new.upTo and possibleRightEnd* > *new.upTo* ensure that a new activation will start only for higher values of upTo and that the cycle will then stop at the rightmost interval.

# DB2 Exam (February 4th, 2022)

A DBMS manages the enrolment of students in the current exam call, using the following two tables:

**ENROLLED STUDENT(<u>StudentID</u>, Name, AssignedRoom)**

**VIRTUAL ROOM(<u>RoomID</u>, NumberOfStudents)**

Table VIRTUAL ROOM keeps a list of virtual rooms with the number of students assigned to them. A virtual room can hold up to 25 students. When a student unsubscribes, the corresponding row is deleted from ENROLLED STUDENT; the system then checks whether it can merge two classes into one with at most 25 students. Virtual rooms with 0 students are deleted.

Write a set of triggers enforcing the above policy when reacting to unsubscribe events.

**ENROLLED STUDENT(<u>StudentID</u>, Name, AssignedRoom)**
**VIRTUAL ROOM(<u>RoomID</u>, NumberOfStudents)**

```
CREATE TRIGGER T1
AFTER DELETE ON ENROLLED_STUDENT
FOR EACH ROW
BEGIN
        UPDATE VIRTUAL_ROOM
        SET NumberOfStudents = NumberOfStudents – 1
        WHERE RoomID = old.assignedRoom;
END
```

*Decrease the NumberOfStudents*

**ENROLLED STUDENT(<u>StudentID</u>, Name, AssignedRoom)**
**VIRTUAL ROOM(<u>RoomID</u>, NumberOfStudents)**

```
CREATE TRIGGER T2
AFTER UPDATE OF NumberOfStudents ON VIRTUAL_ROOM
FOR EACH ROW
WHEN new.NumberOfStudents = 0
BEGIN
        DELETE FROM virtual_room
        WHERE RoomID=new.RoomID;
END
```

*Delete an empty room*

**ENROLLED STUDENT(<u>StudentID</u>, Name, AssignedRoom)**
**VIRTUAL ROOM(<u>RoomID</u>, NumberOfStudents)**

```
CREATE TRIGGER T3
AFTER UPDATE OF NumberOfStudents ON VIRTUAL_ROOM
FOR EACH ROW
WHEN new.NumberOfStudents > 0 AND new.NumberOfStudents < old.NumberOfStudents
BEGIN
        DECLARE R INT;
        SELECT IFNULL((SELECT RoomID FROM VIRTUAL_ROOM
        WHERE NumberOfStudents <= (25 - new.NumberOfStudents)
                AND RoomID<>new.RoomID LIMIT 1), 0) INTO R;
        IF (R<>0) THEN
          UPDATE ENROLLED_STUDENTS SET AssignedRoom=R WHERE AssignedRoom=new.RoomID;
          UPDATE VIRTUAL_ROOM SET NumberOfStudents = NumberOfStudents + new.NumberOfStudents
          WHERE RoomID = R;
          DELETE FROM VIRTUAL_ROOM WHERE RoomID = new.RoomID;
        END IF;
END
```

*Find a room R with enough space*

# DB2 Exam (September1st, 2022)

The following relational schema offers a simplified description of a university (primary keys underlined):

**Person(<u>Pid</u>, Name)**

**Course(<u>Cid</u>, Dept, Credits)**

**Schedule(<u>Teacher, Cid</u>)**

**Registration(<u>Student, Cid</u>)**

with obvious foreign keys from Cid to Course(Cid) and from Teacher and Student to Person(Pid). The university policy requires the following integrity constraints to hold:

- No one is both a student and a teacher for the same course.
- Every course in the CS department is worth at most 10 credits.
- At least one course in the CS department has at least 5 students.

Design a trigger system that keeps the integrity constraints satisfied by rejecting illegal operations.

In particular, the triggers must react to the following kinds of events occurring on the database tables: (i) insertions (any table); (ii) deletions (any table); (iii) updates of Course.

```
CREATE TRIGGER insert_schedule
BEFORE INSERT ON Schedule
FOR EACH ROW
BEGIN
        IF EXISTS( SELECT * FROM Registration
                WHERE new.Teacher = Student AND new.Cid = Cid )
        THEN
                RAISE EXCEPTION 'Existing student with the same name';
        END IF; END;
```

**Person(Pid, Name)**
**Course(Cid, Dept, Credits)**
**Schedule(Teacher, Cid)**
**Registration(Student, Cid)**

```
CREATE TRIGGER insert_registration
BEFORE INSERT ON Registration
FOR EACH ROW
BEGIN
        IF EXISTS( SELECT * FROM Schedule
        WHERE new.Student = Teacher AND new.Cid = Cid )
        THEN
                RAISE EXCEPTION 'Existing teacher with the same name';
        END IF; END;
```

**Person(Pid, Name)**
**Course(Cid, Dept, Credits)**
**Schedule(Teacher, Cid)**
**Registration(Student, Cid)**

```
CREATE TRIGGER delete_registration
BEFORE DELETE ON Registration
FOR EACH ROW
BEGIN
  IF (SELECT dept FROM Course WHERE Cid=old.Cid) = 'CS'
  AND (SELECT COUNT(*) FROM Registration WHERE Cid=old.Cid) = 5
  AND NOT EXISTS(SELECT 1
        FROM Registration R JOIN Course C ON R.Cid=C.Cid
        WHERE dept='CS' AND R.Cid<>old.Cid
        GROUP BY R.Cid HAVING COUNT(*)>=5 )
  THEN
        RAISE EXCEPTION 'No course left with at least 5 students';
END IF;
END;
```

**Person(Pid, Name)**
**Course(Cid, Dept, Credits)**
**Schedule(Teacher, Cid)**
**Registration(Student, Cid)**

```
CREATE TRIGGER update_course_1
BEFORE UPDATE of Credits ON Course
FOR EACH ROW
WHEN new.Credits > 10 AND new.Dept = 'CS'
BEGIN
        RAISE EXCEPTION 'Too many credits for a CS course';
END;
```

**Person(Pid, Name)**
**Course(Cid, Dept, Credits)**
**Schedule(Teacher, Cid)**
**Registration(Student, Cid)**

```
CREATE TRIGGER update_course_2
BEFORE UPDATE of Dept ON Course
FOR EACH ROW
WHEN new.Dept <> 'CS' AND old.Dept = 'CS'
BEGIN
        IF (SELECT COUNT(*)
                FROM Registration WHERE Cid=old.Cid) >= 5
        AND NOT EXISTS( SELECT 1
                FROM Registration R JOIN Course C ON R.Cid=C.Cid
                WHERE Dept='CS' AND R.Cid<>old.Cid
                GROUP BY R.Cid HAVING COUNT(*)>=5 )
        THEN
                RAISE EXCEPTION 'No course left with at least 5 students';
END IF;
END;
```

**Person(Pid, Name)**
**Course(Cid, Dept, Credits)**
**Schedule(Teacher, Cid)**
**Registration(Student, Cid)**

# DB2 Exam (February 17th, 2021)

The following schema describes the Italian National Cashback System for the current semester (Jan-Jun 2021).

User (<u>USERID</u>, Lastname, FirstName, City, NumberOfPayments, TotalCashbackEarned, IsRefundable)

Payment (<u>USERID, DATE, TIME</u>, Amount, Description)

The system grants registered users a 10% cashback reimbursement on all payments, though with some requirements and limitations.

Attribute NumberOfPayments counts the payments done so far in the semester, attribute TotalCashbackEarned contains the (cumulative) cashback earned so far that will be refunded at the end of the semester, and attribute IsRefundable becomes true when the refund gets enabled. Requirements and limitations are defined by the following rules.

The refund is enabled only if the user makes at least 50 payments in the semester, otherwise, the refund is not enabled and no reimbursement occurs.

The 10% cashback earned with a single payment is limited to a maximum of 15 euros (amounts over 150 euros, therefore, gets "saturated").

The total cashback earned cannot exceed 150 euros in the entire semester.

**Design a set of triggers that implement the cashback rules reacting to the insertion of new payments made by the registered users.**

**Users are added to the database upon registration, with the attributes NumberOfPayments, TotalCashbackEarned, IsRefundable initialized to <0, 0, false>: these values are already present when the first payment is received.**

User (USERID, Lastname, FirstName, City, NumberOfPayments, TotalCashbackEarned, IsRefundable)
Payment (USERID, DATE, TIME, Amount, Description)

```
CREATE TRIGGER newpayment
AFTER INSERT ON payment
FOR EACH ROW
BEGIN
UPDATE User
SET NumberOfPayments = NumberOfPayments + 1,
        TotalCashbackEarned = min( 150, TotalCashbackEarned + min( 15,
        0.1*new.Amount ) ),
        IsRefundable = ( 50 <= NumberOfPayments )
        WHERE UserId = new.UserId;
END
```